

CIS-255: Three Join Verification Activities

All three check the same thing: what the join actually returns when it runs in MySQL, not what AI says it returns. All three are built to trigger the professor's exact mistake — picking the wrong join, or joining on the wrong field — and the team has to catch it themselves.

Any organization named below is invented. Any specific number below (percentages, row counts) is a target for whoever builds the dataset, not a prediction about what any team will find.

Idea 1: The 4:58 Ticket

AI's job: the expert who already has the answer.

The situation

It's your first day at Bramwell Outfitters, a fictional online retailer. At 4:58pm your supervisor drops a ticket on your desk: marketing needs a list of every customer who placed an order this month but never got the follow-up email, before a 5:30 send goes out. You've never seen this database before. The AI tool has an answer before you've finished reading the ticket.

Why AI has to be here

A brand-new hire can't produce a first-draft query against an unfamiliar schema in ten minutes — AI turns "I don't know this database" into a starting point instead of a dead stop.

The five steps (50 min)

1. Read the ticket, open MySQL, skim the `customers` and `order_log` tables — **5 min**
2. Ask AI to write the join for "customers who ordered but got no follow-up email," run it as-is — **10 min**
3. Check the result against a printed slip of 6 known customers — should they be on the list or not? — **10 min**
4. Trace which column the query actually joined on — **10 min**
5. Fix the join, rerun, recheck against the same 6 names, write down the corrected count — **15 min**

What lands early

AI hands back a complete, error-free query and a result set in under a minute, against a database the team has never opened. That speed is guaranteed — it's not the surprise. The surprise comes in step 3, when two of the six reference customers don't match, and the team realizes "it ran" isn't the same as "it's right."

Where it goes wrong on purpose

Both tables have a plain `id` column that means nothing outside that table, sitting right next to `customer_id`. AI's cleanest-looking query joins `customers.id = order_log.id` — it runs, it returns rows, it looks done. It's matching unrelated row numbers, not the same customer. The team finds this only by checking the 6-name slip, not by being told.

What only a person can do

Decide which mismatches actually matter to the business — a person has to weigh "customer got the wrong email" against "customer got no email." AI has no basis for that judgment.

What students hand in

The corrected query, plus three sentences naming which column the wrong join used and how they caught it.

What they can do next time

Before trusting a join, check it against a small known-answer sample — never trust the row count alone.

What the data has to contain

- `customers`: `id` (meaningless auto-increment), `customer_id` (the real identifier), `name`, `email`
- `order_log`: `id` (meaningless auto-increment), `customer_id`, `order_date`, `followup_sent` (Y/N)
- Build `customers.id` and `order_log.id` so roughly 40% of rows share the same number by coincidence — this is what makes the wrong join still return a plausible result instead of an obvious error
- Include the 6 reference customers by name, so the professor can hand out a slip without touching the database
- Add 5 `NULL` values in `followup_sent` for orders that predate the tracking system, so "never got it" has a genuine edge case

Prep

Build two small tables (~300 customers, ~500 orders) with the `id` collision planted, write the 6-name slip. About 45 minutes.

Idea 2: Nobody Told It That

AI's job: the beginner they have to train.

The situation

You're a new volunteer coordinator at Larkspur Community Fridge, a fictional neighborhood food-share. The board wants a list of volunteers who haven't taken a shift in 60 days, so someone can call and check in before they're marked inactive. You've never written a query before. You've been told the AI tool will do exactly what you tell it — no more, no less.

Why AI has to be here

A first-time query writer needs something that will literally execute their exact wording and show them the consequence. A human helper would quietly guess and fix their mistakes. AI runs the instructions exactly as given.

The five steps (50 min)

1. Read the request, look at `volunteers` and `shifts` in MySQL — **5 min**
2. As a team, write plain-English, numbered instructions for the join — no SQL yet — **10 min**
3. Hand the instructions to AI exactly as written, get the query, run it — **10 min**
4. Check the result against a printed roster of 25 current volunteers — is everyone who should be flagged actually on the list? — **10 min**
5. Reread the instructions, find the sentence that never said what to do with volunteers who have zero shifts, fix it, rerun — **15 min**

What lands early

AI's first query runs clean, with zero errors, and returns a short, tidy list in seconds. It looks finished. That's guaranteed by the setup — the instructions are complete-sounding even though they're not. The gap only shows up in step 4, against the printed roster.

Where it goes wrong on purpose

The instructions never specify a join type. Left unsaid, AI defaults to an inner join — and volunteers with zero rows in `shifts` (exactly who the board wants to find) get silently dropped, because an inner join only keeps rows that match on both sides. The team discovers this by comparing to the printed roster, not by being told.

What only a person can do

Notice what the instructions didn't say. That's a reading skill, and it's the one thing AI can't do for the team — it can only act on what's written, not what's meant.

What students hand in

The original instructions, the corrected instructions, and one sentence naming the exact gap.

What they can do next time

Always state the join type and the join field out loud — never leave it implied.

What the data has to contain

- `volunteers`: `volunteer_id` (PK), `name`, `join_date`
- `shifts`: `volunteer_id` (FK), `shift_date`
- At least 5 volunteers with zero matching rows in `shifts` — these are the "haven't shown up" cases the activity depends on
- A decoy column: `shifts` also has `lead_id` (the shift supervisor's id), close enough to `volunteer_id` to tempt a wrong-field join if a team's instructions are vague about which id to use
- A handful of `shift_date` values set to `NULL` (shift was scheduled, then cancelled), so "no shift" and "cancelled shift" aren't the same thing

Prep

Build two tables (~60 volunteers, ~400 shift rows), plant the 5 zero-shift volunteers, print the 25-name roster slip. About 30 minutes.

Idea 3: The Shortlist

AI's job: the drafting room that never tires.

The situation

You're interning at Pemberton Freight, a fictional regional trucking company. Dispatch wants one query, to run every morning, joining `shipments` to `drivers` to flag deliveries running more than a day late. Your supervisor says "just get me something that works" and walks off. You've never written this join before, and you're not sure there's only one right way to do it.

Why AI has to be here

A team of four in 50 minutes can't write and test ten full query drafts by hand. AI makes producing that many versions nearly free, so the real work of the class becomes comparing them, not writing them one at a time.

The five steps (50 min)

1. Read the request, look at `shipments` and `drivers` together as a team — **5 min**
2. Ask AI for 8 different ways to write the join for "late deliveries by driver" — **10 min**
3. Run all 8 against the real database, one at a time, log the row count each returns — **15 min**
4. Group the 8 by matching count; for one query from each group, find the exact line where it differs — **10 min**

5. Check which group's result contains all 5 shipments on a professor-supplied "known late" slip, and pick that version — **10 min**

What lands early

AI hands back 8 complete, ready-to-run drafts in under a minute — different aliases, different clause order, different join syntax. The surprise isn't that AI can write one join; it's that it writes eight without being asked twice, and every one of them looks equally confident. Guaranteed by asking for 8 up front.

Where it goes wrong on purpose

`drivers` has a `driver_id` column and a free-text `driver_name` column with inconsistent formatting (extra spaces, mixed case — "J. Ortiz" vs "j.ortiz "). Several of AI's 8 drafts join on `driver_name` instead of `driver_id` because it reads as a plausible shared field. Those versions silently miss any shipment where the name formatting doesn't match exactly, undercounting late deliveries. The team finds this only by comparing row counts and the 5 known shipments across groups.

What only a person can do

Decide which of several technically-working queries the business should actually run every morning. That's a judgment about which mistake is worse — missing a late shipment, or flagging one that isn't — and no version of AI's output settles that on its own.

What students hand in

The chosen query, the row count for each of the 8 versions, and one sentence on why the rejected versions were wrong.

What they can do next time

Never accept the first version that runs — test the same request more than one way and see if the results agree before trusting any of them.

What the data has to contain

- `drivers`: `driver_id` (PK), `driver_name` (free text, deliberately inconsistent — extra whitespace and mixed case on ~15% of rows)
- `shipments`: `shipment_id`, `driver_id`, `driver_name_copy` (a denormalized text copy of the driver's name, so a name-based join is even possible), `scheduled_date`, `delivered_date`
- At least 30 shipments where `delivered_date` is more than 1 day after `scheduled_date` — the actual "late" population the correct query has to catch
- The 5 known-late shipments on the answer slip must be a subset of those 30, and at least 2 must belong to a driver with inconsistent name formatting — this is what makes the name-based join drop them

Prep

Build two tables (~40 drivers, ~500 shipments), plant the inconsistent formatting and the 30 late shipments, write the 5-shipment answer slip. About 45 minutes.